Technical Challenge: Periodic Student Data Ingestion Pipeline

Scenario: A school district receives student information from multiple sources, including scheduled CSV exports and third-party APIs. The organization needs a reliable ingestion platform that periodically collects, validates, transforms, and writes student records into a database for reporting and downstream application use.

Challenge Objective

Design and implement a production-minded data ingestion solution that can periodically ingest student data from CSV files and APIs, normalize the incoming data, handle failures safely, and persist the final records into a relational or document database hosted on Azure

Input Data

- **CSV source:** A periodically delivered file containing student records such as student_id, first_name, last_name, grade_level, school_id, email, enrollment_status, updated_at, and guardian_contact.
- **API source:** A REST API that returns incremental student updates, enrollment status changes, or supplemental profile data.
- **Volume assumptions:** The solution should support small daily files as well as larger batch loads without requiring a full redesign.

Core Requirements

1. Ingest CSV files from a storage location or upload endpoint on a configurable schedule.
2. Call one or more external APIs on a configurable schedule, supporting authentication, pagination, retries, and rate-limit handling.
3. Validate required fields, data types, duplicate student identifiers, invalid email formats, and malformed records.
4. Transform source data into a canonical student schema before persistence.
5. Write valid records to a database using idempotent upsert behavior.

Cloud Hosting Requirement

Candidates should propose a practical way to host and operate the ingestion solution on a cloud platform such as Azure, or an equivalent provider. The proposal should explain the selected compute, storage, database, scheduling, monitoring, security, and deployment approach, along with key trade-offs.

Expected Deliverables

- Architecture diagram showing ingestion sources, processing components, database, monitoring, and failure handling.
- Source code or pipeline definitions for CSV ingestion, API ingestion, transformation, validation, and database writes.
- Database schema or collection design, including primary keys and indexes.
- Infrastructure-as-code template using Terraform or similar tooling.
- README with setup instructions, assumptions, local testing steps, cloud deployment steps, and trade-offs.
- Sample input files, sample API response payloads, and expected output examples.
- Operational notes describing retries, idempotency, logging, alerting, and reprocessing strategy.

Evaluation Criteria

Area	What to Look For
Architecture	Clear separation of ingestion, validation, transformation, persistence, and monitoring responsibilities.
Data correctness	Strong validation rules, duplicate handling, deterministic transformations, and idempotent writes.
Cloud readiness	Appropriate use of managed services, least-privilege access, secure secrets handling, and deployable infrastructure.
Reliability	Retry strategy, dead-letter or quarantine handling, observability, and safe reprocessing.
Scalability	Design can grow from small scheduled files to larger datasets or more frequent API polling.

Maintainability	Readable code, modular pipeline design, tests, documentation, and clear assumptions.